Guided Assignment 4

Hints and worked solutions

Use sparingly. Attempt each stage first. This page contains hints only, and worked answers start on the next page. After checking an answer, close this file and reconstruct the argument yourself. Figures come from a numpy LIF *rate* model (no Nengo, no spikes). They illustrate mechanisms but do not reproduce your notebook runs exactly.

One hint per stage

1. Invert $A' = \tau A + I$, $B' = \tau B$. For saturation, ask where the decoders were fitted and what $\hat{f}(x)$ does outside that region; equilibrium is where $\dot{x} = 0$.

2. Integrate polynomials directly. To expand r^2, first solve $\tilde{P}_2$ for r^2, then substitute your expansion of r.

3. For $q = 1$ there is a single entry with $i = j = 0$. For delays, evaluate each $\tilde{P}_i$ at θ'/θ. Count the cycles that fit in 0.5 s.

4. Differentiate $\frac{1}{2}\varepsilon^2$ with respect to d_j; only one term of the sum depends on d_j. Then substitute into $w_{ij} = \alpha_i \langle \mathbf{e}_i, \mathbf{d}_j \rangle$.

5. At equilibrium every derivative is zero, so the torques balance. The PD output is whatever torque is still missing.

6. Firing rates are non-negative. Which term in each modified rule can make Δw negative?

7. U is orthogonal, so $UU^T = I$ lets you factor $\sigma^2 I$ inside $U(\cdot)U^T$. For the noise derivation, use orthogonality to kill the cross terms.

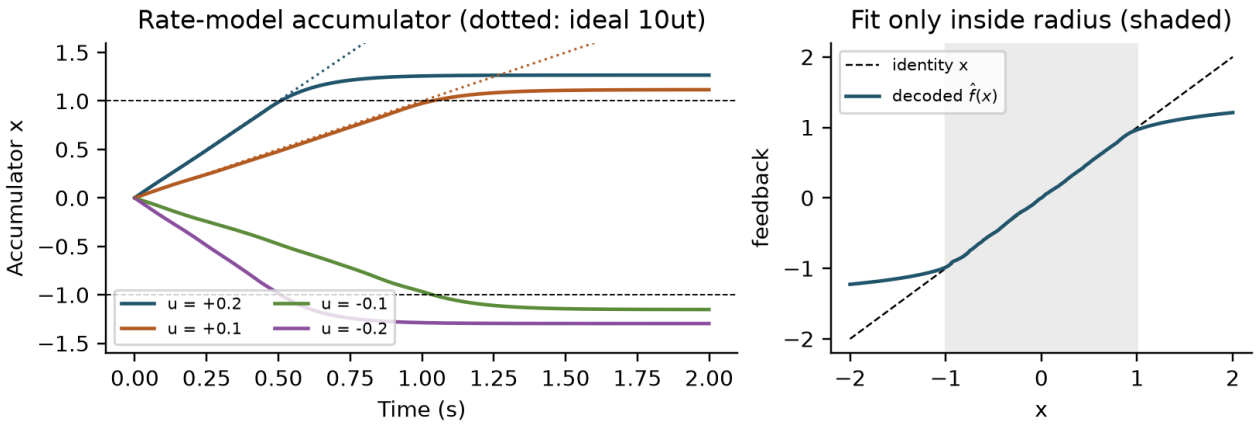
8. No new techniques. Return to the matching stage for any item that stalls.

1. Accumulate-to-threshold decision model

1a. $A' = 1 \Rightarrow A = 0$, and $B' = 1 \Rightarrow B = 1/\tau = 10$ s^{-1}. So $\dot{x} = 10u$ and $x(t) = 10ut$. The network *integrates* the evidence: its output is proportional to evidence × time. Because the input was not pre-scaled by τ, it integrates ten times faster than $\dot{x} = u$. The default input synapse is 5 ms, not 0.1 s, so this mapping is approximate. That mismatch mainly changes the onset transient.

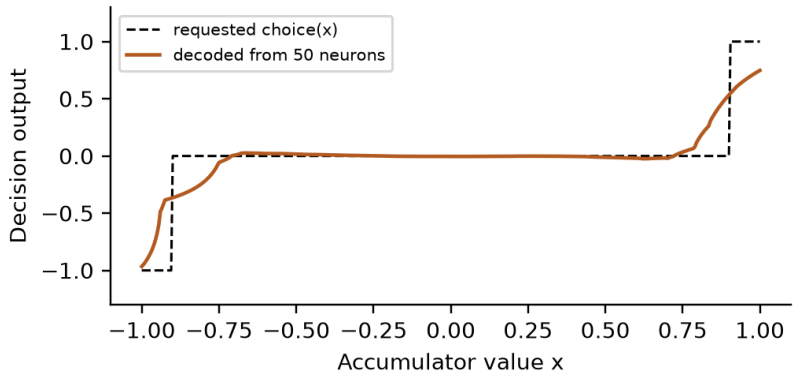
1b. $t^* = 0.9/(10|u|) = 0.09/|u|$, giving 0.9, 0.45 and 0.3 s for $u = 0.1, 0.2, 0.3$. The plot is a hyperbola: decision time is inversely proportional to evidence strength. Halving the evidence doubles the time. This is the defining behavioural signature of accumulate-to-threshold models.

1c. Inside $[-1, 1]$, $\hat{f}(x) \approx x$. Outside, $\hat{f}$ bends toward horizontal, so $|\hat{f}(x)| < |x|$. No evaluation points lie beyond the radius, so nothing forces the fit there. LIF rates flatten (they are concave and bounded by $1/\tau_{ref}$), and no neurons have intercepts beyond 1 to be newly recruited. The equilibrium of $\tau\dot{x} = \hat{f}(x) - x + u$ is where the gap $x - \hat{f}(x)$ equals u: the leak created by under-represented feedback balances the input. For $u = 0.2$ this lies slightly beyond 1 (about 1.25 in the illustration). Stronger input settles slightly further out. The traces are ramps with slopes ±1 and ±2 per second that level off just beyond ±1.



1d. The model becomes $\dot{x} = 10u + \xi(t)$, a random walk with drift, also called a *drift-diffusion* model. Increasing noise (i) makes traces jagged around the ramp; (ii) spreads decision times, producing a right-skewed distribution, and noise can cross the threshold earlier *or* later than the ideal t^*; (iii) causes errors, because weak evidence ($u = 0.1$) sometimes reaches -0.9 first. At very large noise (RMS 10), the stored value is dominated by noise and decisions become close to chance. Noise also jitters the saturated value near ±1.

1e. The decoded curve (i) rises smoothly and *early*, from about 0.75 rather than at 0.9; (ii) under-reaches ±1, reaching only about 0.75 at $x = 1$ in the illustration; and (iii) shows small ripples and nonzero values in the middle. A finite sum of continuous tuning curves is continuous and cannot jump. A sharp step needs high-order, high-frequency components, and those have small singular values (Stage 7), so regularization suppresses them.



1f. The $e = +1$ neurons are zero until their intercept (0.4–0.9), then rise steeply to their maximum at $y = 1$. The $e = -1$ neurons are mirror images. (i) When $|y| < 0.4$, every neuron is silent, and the decoder is linear with no constant term, so $\hat{y} = \sum d_i \cdot 0 = 0$ *exactly*. Small ripples from the choice decoding fall inside this dead zone. (ii) All neurons are devoted to $|y| > 0.4$, so the population's representational resources concentrate where ± 1 lives. The output also snaps on sharply once the drive passes the intercepts. (iii) Values with $|y| < 0.4$ cannot be represented; they all read as 0, and intermediate values are distorted. This is acceptable only because the output should be 0 or ± 1.

1g. The joint vector (x, y) reaches the corners $(\pm 1, \pm 1)$ with norm $\sqrt{2}$, so the radius must be at least $\sqrt{2}$. At 100 neurons, accuracy is worse: there are fewer neurons per dimension, a 2-D space must be tiled, and absolute error grows with radius. At 500 neurons it improves. The intercept trick fails because an intercept is a threshold on $\mathbf{e} \cdot \mathbf{v}/R$ for the *whole* vector. High intercepts would silence neurons over most of the space, including small accumulator values, and you cannot make a neuron blind near zero in only the decision coordinate. The biological claim changes from separate evidence and decision populations to the same neurons carrying both, which is *mixed selectivity*. One test: record single neurons during the task and ask whether individual cells' rates depend on accumulated evidence *and* on the eventual choice, or whether cells divide cleanly into the two roles. Selective inactivation that impairs one variable but not the other would also argue for separation.

2. Representing a function as a vector

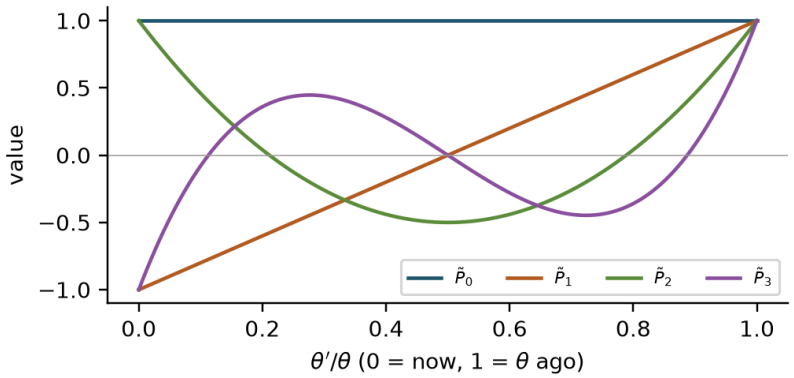
2a. A signal with no frequencies above B is completely determined by samples spaced $1/(2B)$ apart, so $N = 2BT$. A 2 Hz signal over a 0.5 s window needs $N = 2$ samples; a 5 Hz signal over 3 s needs $N = 30$. With delta basis functions $\phi_i = \delta(t - i\Delta t)$, the coefficient $\langle \phi_i, f \rangle = f(i\Delta t)$ is exactly the sample.

2b. $\int_0^1 (2r - 1)\, dr = [r^2 - r]_0^1 = 0$ and $\int_0^1 (2r-1)^2 dr = \frac{4}{3} - 2 + 1 = \frac{1}{3}$. The basis is orthogonal but not orthonormal: in general $\|\tilde{P}_i\|^2 = 1/(2i + 1)$.

2c. $r = \frac{1}{2}\tilde{P}_0 + \frac{1}{2}\tilde{P}_1$. From $\tilde{P}_2 = 6r^2 - 6r + 1$, $r^2 = \frac{1}{6}\tilde{P}_2 + r - \frac{1}{6}$, so

$$r^2 = \tfrac{1}{3}\tilde{P}_0 + \tfrac{1}{2}\tilde{P}_1 + \tfrac{1}{6}\tilde{P}_2.$$

Check: $r = 1$ gives $\frac{1}{3} + \frac{1}{2} + \frac{1}{6} = 1$, and $r = 0$ gives $\frac{1}{3} - \frac{1}{2} + \frac{1}{6} = 0$. A $q = 2$ representation drops $\frac{1}{6}\tilde{P}_2$, leaving $r - \frac{1}{6}$, which is the best straight-line fit to r^2 on $[0, 1]$.



2d. *Representing* a function means the population's state stands for an entire function, here a window of the past, through its coefficients. *Computing* means a connection maps a represented value x to $f(x)$. When decoders are solved, the tuning curves $a_i(x)$ are the basis functions and the decoders d_i are the least-squares coefficients that combine them into the target $f(x)$. It is the same problem as finding $\mathbf{x}$ for a non-orthogonal basis.

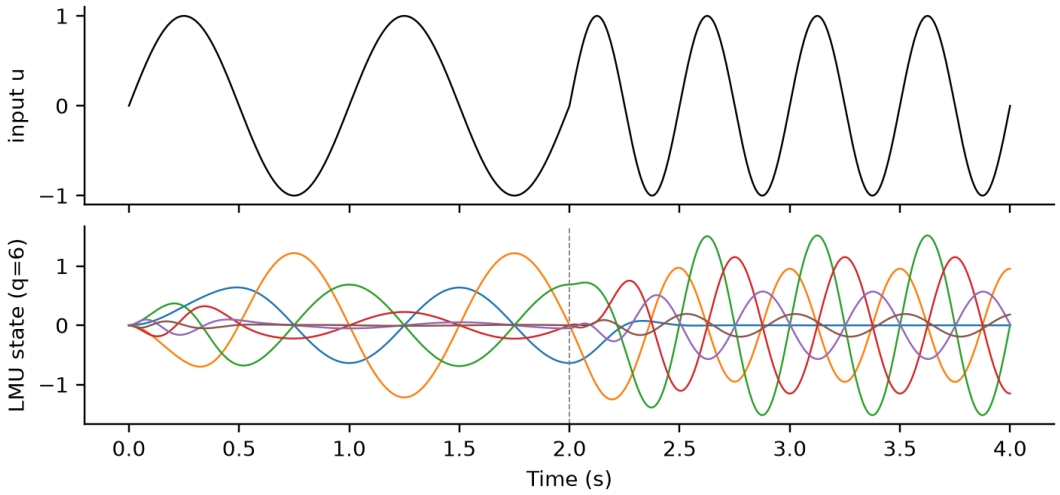
3. The Legendre Memory Unit

3a. $A = -1/\theta$ and $B = 1/\theta$, so $\dot{x} = (u - x)/\theta$: a first-order *low-pass* filter, the [0/1] Padé approximation $1/(1+\theta s)$ of $e^{-s\theta}$. x is an exponentially weighted average of roughly the last θ seconds. $A' = 1 - \tau/\theta = 0.8$ and $B' = \tau/\theta = 0.2$. Step response: $x = 1 - e^{-t/\theta}$, which is already 0.63 at $t = \theta$. A true delay is 0 until $t = 0.5$ and then jumps to 1, so $q = 1$ gives a smeared, early version of the delay.

3b. $\theta A = \begin{pmatrix} -1 & -1 \\ 3 & -3 \end{pmatrix}$ and $\theta B = \begin{pmatrix} 1 \\ -3 \end{pmatrix}$. With $\tau/\theta = 0.2$, $A' = 0.2\,\theta A + I = \begin{pmatrix} 0.8 & -0.2 \\ 0.6 & 0.4 \end{pmatrix}$ and $B' = \begin{pmatrix} 0.2 \\ -0.6 \end{pmatrix}$. In general A' is $q \times q$ and B' is $q \times 1$.

3c. *Now* ($\theta' = 0$): weights $(-1)^i = (1, -1, 1, -1)$, so $u(t) \approx x_0 - x_1 + x_2 - x_3$. θ *ago*: weights $(1, 1, 1, 1)$, so $u(t - \theta) \approx x_0 + x_1 + x_2 + x_3$. *Middle*: $\tilde{P}_0 = 1$, $\tilde{P}_1(\frac{1}{2}) = 0$, $\tilde{P}_2(\frac{1}{2}) = \frac{3}{2} - 3 + 1 = -\frac{1}{2}$ and $\tilde{P}_3(\frac{1}{2}) = 0$, giving $(1, 0, -\frac{1}{2}, 0)$; this was confirmed numerically. A pure delay must eventually output every value currently inside the window, so all of it must be stored in the state.

3d. 0.5 s holds half a cycle of 1 Hz and one full cycle of 2 Hz. The window average x_0 is therefore about 0 for 2 Hz, because a full cycle averages out. For 1 Hz it oscillates with amplitude about $2/\pi \approx 0.64$, the average of a half sine. The 2 Hz energy moves into higher-order coefficients. Both sines visit every value in $[-1, 1]$, so $u(t)$ alone is ambiguous, but the 6-D state traces different trajectories for the two classes. The figure shows the ideal (non-neural) LMU state; note the change at $t = 2$ s.



3e. (1) Near $t = 0$ the window is mostly empty, and the 0.1 s synapse adds lag. The state is close to zero and carries little frequency information until enough of the signal has entered; the two sines first differ clearly after about 0.125–0.25 s. (2) The classifier saw the start-up states only with the label +1, because 1 Hz came first. Start-up states therefore map to +1, which is wrong when 2 Hz comes first. Fixes: train on both orders or random orders; exclude the first θ seconds from training, or give them a neutral target; warm-start the window; or accept a short "undecided" period.

3f. *Neurons:* RMSE falls steeply from 50 neurons, which is far too few for a 6-D state, with diminishing returns at large counts (the noise part scales roughly as $1/\sqrt{N}$). It levels off at a floor set by the classifier and the finite window. *q:* U-shaped. Small q cannot describe the window's shape; large q spreads the neurons over many dimensions, and the high-order coefficients have large gains $(2i+1)/\theta$ and fast dynamics that the 0.1 s synapses and the radius cannot implement. The state components reach about 1.5 for a unit sine, so a radius-1 ensemble would saturate them (compare Stage 1). Scale the input or the radius. In high dimensions, $\mathbf{e} \cdot \mathbf{x}$ for random unit vectors concentrates near 0. With uniform intercepts, many neurons would then be always on or always off; `CosineSimilarity(q+2)` matches the intercepts to that distribution.

4. Online learning with PES

4a. Reasons to learn: the function may not be known in advance (which food is edible?); the required function may change (injury, growth, new loads); the offline D is optimal for decoders and static rates, not for the full $W = ED$ or the spiking dynamics; and we want to study learning itself. Integrator memory, the delay network or firing-rate adaptation are adaptive, but they are not learning under this definition.

4b. $\partial E/\partial d_j = (\sum_i d_i a_i - y^d)\, a_j = \varepsilon a_j$, so $\Delta d_j = -\eta\varepsilon a_j$. If neuron j is active while the output is too high ($\varepsilon > 0$), d_j decreases, which reduces its push on the output. Silent neurons are not changed.

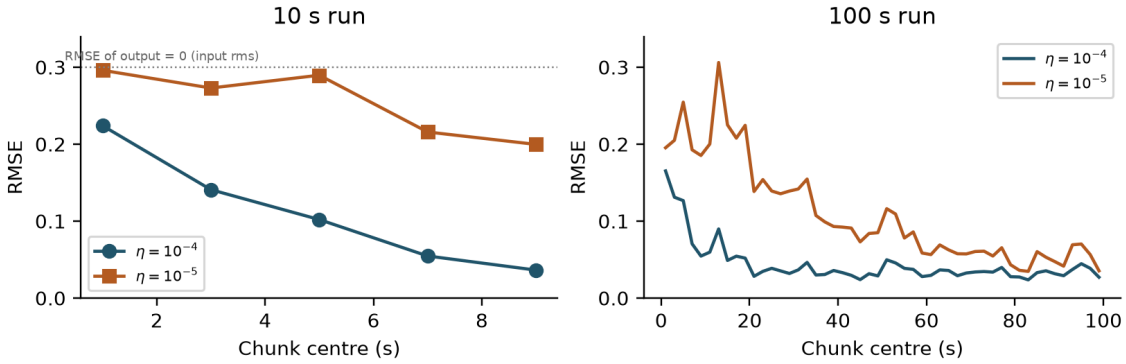
4c. $\Delta w_{ij} = \alpha_i \langle \mathbf{e}_i, \Delta \mathbf{d}_j \rangle = -\eta\, \alpha_i \langle \mathbf{e}_i, \boldsymbol{\varepsilon} \rangle\, a_j$. The presynaptic factor is a_j. The postsynaptic, modulatory factor is $\alpha_i \langle \mathbf{e}_i, \boldsymbol{\varepsilon} \rangle$: the error encoded into neuron i as if it were an input current, comparable to a dopaminergic signal. The rule is local, because each synapse needs only its pre-activity and a signal delivered to its post-neuron. Decoders do not physically exist in the brain.

4d. stim → pre (identity) → post (learned connection: initial function 0, PES). post → error with +1, stim → error with −1, so error = output − target. Error → the connection's learning rule. For $[x^2, -x]$: post and error become 2-D, pre stays 1-D, and the stim → error connection gets `function=lambda x: [x**2, -x]` with transform −1 (or a function returning the negative). The learned connection outputs

2-D.

4e. The rule then performs gradient *ascent*. A too-large output is pushed larger, which is positive feedback, so the output runs away from the target until saturation and the error grows.

4f. 10 s: the RMSE falls across the chunks and levels off above zero. 100 s: it falls within the first 10–20 s, then fluctuates around a lower floor. At $\eta = 10^{-5}$ learning is about ten times slower. After 100 s it approaches the floor the faster rate reached early, and with long training a small η can give a quieter floor because the weights jitter less. Early progress scales roughly with $\eta\times$ time. The floor comes from finite neurons, noisy pre and error signals, ongoing weight jitter and filter lag. Improvements: more neurons in pre and error; a lower or annealed learning rate with longer training; inputs that cover the whole domain; a filtered error; or starting from offline decoders instead of zero.



4g. Only neurons active for $x \in [0.5, 1]$ change. Many of them, those with positive encoders and negative intercepts, also fire for some negative x. Fitting the positive region therefore changes their contribution elsewhere, and no error arrives from the negative region to correct it. The mapping there drifts. This is catastrophic forgetting: the network over-fits the current region of input space.

4h. The unconditioned response, driven by the US, acts as the teaching signal. The error compares the response predicted from the CS with the UR, so the error is generated inside the system and no external teacher is needed. During extinction the CS appears without the US, the target is zero, and the error weakens the learned CS $\rightarrow$ response mapping.

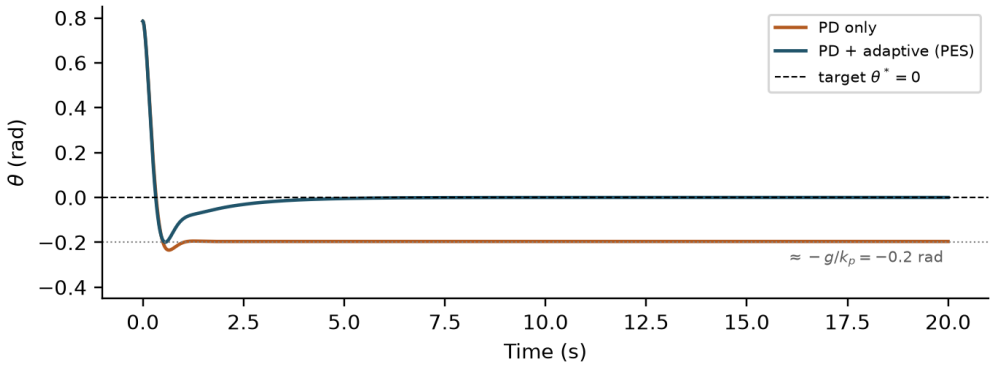
5. Adaptive control of a pendulum

5a. At equilibrium $k_p(\theta^* - \theta) = mgL\cos\theta$. Write $\theta = \theta^* + \delta$ with $\cos(\theta^* + \delta) \approx \cos\theta^*$; then $\delta \approx -mgL\cos\theta^*/k_p$. For $\theta^* = 0$ and $k_p = 49$, $\theta_{ss} \approx -0.2$ rad, below the horizontal. For $\theta^* = \pi/2$, $\cos = 0$: gravity acts along the vertical arm and exerts no torque, so there is no offset. A huge k_p shrinks the offset only as $1/k_p$. It amplifies sensor noise, demands large torques that saturate actuators, reduces stability with delays, and still does not adapt to changing loads.

5b. $u_{ad}(\theta) \approx mgL\cos\theta$, plus any other unmodelled terms. The required torque depends on the angle. A constant input could learn only one torque value, correct at one angle, so the ensemble must represent the state.

5c. At equilibrium $0 = \tau_{PD} + u_{ad} - mgL\cos\theta$, so $\tau_{PD} = mgL\cos\theta - u_{ad}$. Learning that drives $\tau_{PD} \rightarrow 0$ therefore drives $u_{ad} \rightarrow mgL\cos\theta$, and the offset vanishes. Sign: the adaptive torque should grow when the PD output is positive. Since $\Delta d = -\eta\varepsilon a$, use $\varepsilon = -\tau_{PD}$, that is, connect the PD signal to the learning rule with transform -1.

5d. PD alone: the angle falls from 0.79 with a small undershoot and holds at -0.2. Adaptive: the same transient, then the offset decays to 0 over seconds. After a switch to $\pi/4$: the learned function is accurate mainly near states visited during training ($\theta \approx -0.2$ to 0). Some offset reappears, smaller than with PD alone, and is then quickly relearned. A bias-only learner would supply the full 9.8 where only $9.8\cos(\pi/4) \approx 6.9$ is needed, and would overshoot *above* the target.



5e. Same: PES gradient descent on decoders from a state ensemble, driven by an error. Different: there is no explicit target function. The error is generated by the controller, and the learned output acts through the plant's dynamics.

6. Unsupervised learning

6a. $\Delta w_{ij} = \eta a_i a_j$. Rates are non-negative, so $\Delta w \geq 0$ always. Larger weights produce more post activity and therefore larger updates, so the weights run away. The rule counts coincident activity only and ignores which neuron caused the other's firing.

6b. Oja: $\Delta w_{ij} = \eta(a_i a_j - a_i^2 w_{ij})$. The decay term, proportional to the squared post activity, keeps $\|\mathbf{w}_i\| \to 1$, and $\mathbf{w}_i$ converges to the first principal component of the pre-activity. BCM (as written in Lecture 8): $\Delta w_{ij} = \eta a_j a_i(\theta - a_i)$, which grows when the post rate is below the target θ and shrinks above it, a homeostatic rule. Standard BCM papers use the opposite sign with a sliding threshold; use the lecture's version on the test.

6c. Pre before post ($\Delta t > 0$) gives potentiation, $\alpha^+ e^{-\Delta t/\tau^+}$. Post before pre ($\Delta t < 0$) gives depression, $-\alpha^- e^{+\Delta t/\tau^-}$. Both sides decay with $|\Delta t|$ over about 10 ms. The notes print $e^{-\theta/\tau^-}$ for $\theta \leq 0$, which would grow without bound; that is a typo.

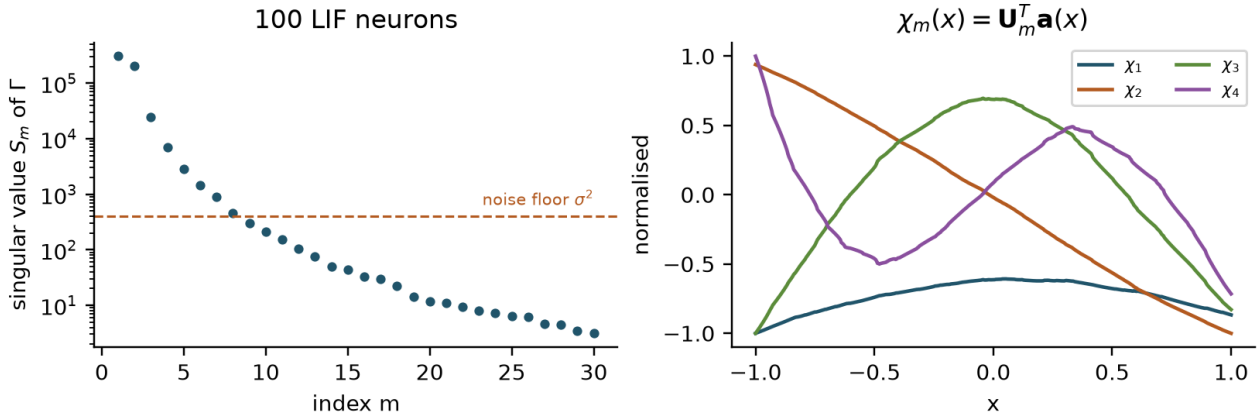
6d. PCA uses the eigenvectors of the covariance matrix of the centred data. With $X_c = USV^T$, $C = V\frac{S^2}{N-1}V^T$, so V holds the principal directions. Limitation: PCA is linear. It cannot capture curved structure, and maximum-variance directions need not separate groups.

7. Analysing representation with the SVD

7a. In Lecture 9's convention (A is $M \times N$), $D = (A^T A)^{-1} A^T X$, so $\Gamma = A^T A$ is $N \times N$ and $\Upsilon = A^T X$ is $N \times d$. Every target changes only Υ; Γ is common to all decodings, so its eigen-structure sets which target directions are resolved well. A large S_m means: high variance of population activity along U_m; a large error if that direction is left out; and more independent information that is robust to noise.

7b. $\Gamma^{-1} = (USU^T)^{-1} = US^{-1}U^T$, using $U^{-1} = U^T$. Also $\Gamma + \sigma^2 I = USU^T + \sigma^2 UU^T = U(S + \sigma^2 I)U^T$, so its inverse is $U(S+\sigma^2 I)^{-1}U^T$. A tiny S_m no longer produces a huge $1/S_m$; the factor is capped at $1/\sigma^2$. The pseudo-inverse sets $1/S_m \to 0$ for $S_m = 0$, ignoring null directions and giving the minimum-norm solution.

7c. $\hat{X} = AD = AUS^{-1}U^T A^T X = (AU)(S^{-1}U^T A^T X) = \chi\Phi$, and $U^T D = S^{-1}U^T A^T X = \Phi$. The spectrum decays fast, roughly exponentially, then has a flat tail. $\chi_1, \chi_2, \chi_3, \chi_4$ look like constant, linear, quadratic and cubic Legendre polynomials (signs are arbitrary). Gaussian bumps give a Fourier-like (sinusoidal) basis instead. With clumped intercepts the polynomial ordering breaks; for example, the linear term is no longer better supported than the quadratic one.



7d. $\partial E/\partial\Phi_n = -2\langle(x - \sum_m \chi_m\Phi_m)\chi_n\rangle + 2\sigma^2\Phi_n = -2c_n + 2S_n\Phi_n + 2\sigma^2\Phi_n = 0$. Hence $\Phi_n = c_n/(S_n + \sigma^2)$ with $c_n = \langle\chi_n x\rangle$. If $S_n \gg \sigma^2$, $\Phi_n \approx c_n/S_n$ and noise does not matter. If $S_n \lesssim \sigma^2$, the coefficient shrinks toward zero and the mode contributes little: its error reduction is $c_n^2/(S_n + \sigma^2)$. Rule: only modes with $S_m > \sigma^2$ are usable, so the noise sets the representation's effective dimensionality. (Using the noise-free c_n/S_n for such a mode would *add* error; that is Practice Test Question 5(c).)

7e. Order 0: 1. Order 1: x_1, x_2. Order 2: x_1^2, x_1x_2, x_2^2. Order 3: four terms. That gives $1, 2, 3, 4 = l + 1 = \binom{l+1}{l}$. For $D = 3$, $l = 2$: $\binom{4}{2} = 6$ terms $(x^2, y^2, z^2, xy, xz, yz)$. There are more terms at every order and their singular values shrink with order, so more of the needed modes fall below the noise. High-order functions of high-dimensional vectors need many more neurons.

7f. (i) A step requires high-order modes whose S_m lie below the noise floor. Regularization drops them, so the decoded step has rounded corners and under-reaches. (ii) Averaged PES gives $\dot{\Phi}_m = -\eta(S_m\Phi_m - \langle\chi_m y^d\rangle)$, so each mode converges with time constant $1/(\eta S_m)$. x uses the large-S linear mode; x^2 needs the smaller-S quadratic mode and is learned more slowly. (iii) Heterogeneous tuning tiles the space, preserves the low-order-first ordering, keeps the low-order singular values large (robust to noise), and is cheap to build.

8. Readiness check: brief answers

8a. $B = 1/0.05 = 20$ s^{-1}, $x = 20ut$, so $t^* = 0.9/(20 \cdot 0.3) = 0.15$ s. The trace ramps, then levels off just beyond 1. **8b.** $\dot{x} = (u - x)/\theta$ with $A' = 1 - 0.05 = 0.95$ and $B' = 0.05$. It holds an exponentially weighted average of about the last second. **8c.** $\Delta d_j = -\eta\varepsilon a_j$. For the target $-x$: error $= \hat{y} - (-x) = \hat{y} + x$, so both post $\to$ error and stim $\to$ error use transform $+1$. **8d.** The offset is about $-9.8/98 = -0.1$ rad. The traces match Stage 5 with half the offset. **8e.** $\Phi_n = \langle\chi_n x\rangle/(S_n + \sigma^2)$. The population decodes smooth,

low-order functions well (linear, quadratic, roughly cubic). It does poorly on sharp steps, high-frequency and high-order functions.